In [2]:
```
import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv("Telco-Customer-Churn.csv")

print("Dataset imported successfully!")
print("Shape:", df.shape)

df.head()
```

```
Dataset imported successfully!
Shape: (7043, 21)
```

Out[2]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	In
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	
4	9237-HQITU	Female	0	No	No	2	Yes	No	

5 rows × 21 columns

In [3]:
```
# Display first 5 records
print("First 5 records:")
display(df.head())

# Display last 5 records
print("\nLast 5 records:")
display(df.tail())

# Dataset shape
print("\nDataset Shape:", df.shape)

# Column names
print("\nColumn Names:")
print(df.columns.tolist())

# Dataset information
print("\nDataset Information:")
df.info()

# Statistical summary
```

```
print("\nStatistical Summary:")
display(df.describe(include="all"))
```

First 5 records:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Inter
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	
4	9237-HQITU	Female	0	No	No	2	Yes	No	

5 rows × 21 columns

Last 5 records:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLir
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Y
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Y
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No pho servi
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Y
7042	3186-AJIEK	Male	0	No	No	66	Yes	N

5 rows × 21 columns

```
Dataset Shape: (7043, 21)

Column Names:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure',
'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity',
'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV',
'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod',
'MonthlyCharges', 'TotalCharges', 'Churn']

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

Statistical Summary:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	
count	7043	7043	7043.000000	7043	7043	7043
unique	7043	2	NaN	2	2	
top	7590-VHVEG	Male	NaN	No	No	
freq	1	3555	NaN	3641	4933	
mean	NaN	NaN	0.162147	NaN	NaN	32
std	NaN	NaN	0.368612	NaN	NaN	24
min	NaN	NaN	0.000000	NaN	NaN	0
25%	NaN	NaN	0.000000	NaN	NaN	9
50%	NaN	NaN	0.000000	NaN	NaN	29
75%	NaN	NaN	0.000000	NaN	NaN	55
max	NaN	NaN	1.000000	NaN	NaN	72

11 rows × 21 columns

In [4]:
```
# Check missing values before cleaning
print("Missing values before cleaning:")
print(df.isnull().sum())

# Convert TotalCharges to numeric
# Invalid/blank values will become NaN
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

# Check missing values again
print("\nMissing values after converting TotalCharges:")
print(df.isnull().sum())
```

```
Missing values before cleaning:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Missing values after converting TotalCharges:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges       11
Churn               0
dtype: int64
```

In [5]:

```python
# Fill missing numeric values with median
numeric_columns = df.select_dtypes(include=np.number).columns

for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())

# Fill missing categorical values with mode
categorical_columns = df.select_dtypes(include="object").columns

for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```
print("Missing values after handling:")
print(df.isnull().sum())
```

```
Missing values after handling:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

In [6]:
```
# Count duplicate records
print("Number of duplicate records:", df.duplicated().sum())

# Remove duplicates
df = df.drop_duplicates()

print("Shape after removing duplicates:", df.shape)
```

```
Number of duplicate records: 0
Shape after removing duplicates: (7043, 21)
```

In [7]:
```
# Get categorical columns
categorical_columns = df.select_dtypes(include="object").columns

# Remove leading/trailing spaces
for col in categorical_columns:
    df[col] = df[col].astype(str).str.strip()

# Standardize Gender
if "gender" in df.columns:
    df["gender"] = df["gender"].str.capitalize()

# Display unique values of categorical columns
for col in categorical_columns:
    print("\n", col)
    print(df[col].unique())
```

```
 customerID
['7590-VHVEG' '5575-GNVDE' '3668-QPYBK' ... '4801-JZAZL' '8361-LTMKD'
 '3186-AJIEK']

 gender
['Female' 'Male']

 Partner
['Yes' 'No']

 Dependents
['No' 'Yes']

 PhoneService
['No' 'Yes']

 MultipleLines
['No phone service' 'No' 'Yes']

 InternetService
['DSL' 'Fiber optic' 'No']

 OnlineSecurity
['No' 'Yes' 'No internet service']

 OnlineBackup
['Yes' 'No' 'No internet service']

 DeviceProtection
['No' 'Yes' 'No internet service']

 TechSupport
['No' 'Yes' 'No internet service']

 StreamingTV
['No' 'Yes' 'No internet service']

 StreamingMovies
['No' 'Yes' 'No internet service']

 Contract
['Month-to-month' 'One year' 'Two year']

 PaperlessBilling
['Yes' 'No']

 PaymentMethod
['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
 'Credit card (automatic)']

 Churn
['No' 'Yes']
```

In [8]:
```
# Columns containing Yes/No type values
yes_no_columns = [
    "Partner",
    "Dependents",
    "PhoneService",
    "PaperlessBilling",
    "Churn"
```

```
]

for col in yes_no_columns:
    if col in df.columns:
        df[col] = df[col].replace({
            "yes": "Yes",
            "no": "No",
            "YES": "Yes",
            "NO": "No"
        })

print("Categorical values standardized successfully.")
```

```
Categorical values standardized successfully.
```

In [9]:

```
# Convert numeric columns
numeric_columns = [
    "SeniorCitizen",
    "tenure",
    "MonthlyCharges",
    "TotalCharges"
]

for col in numeric_columns:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

# Convert Churn from Yes/No to 1/0
if "Churn" in df.columns:
    df["Churn"] = df["Churn"].map({
        "Yes": 1,
        "No": 0
    })

print("Data types after conversion:")
print(df.dtypes)
```

```
Data types after conversion:
customerID           object
gender               object
SeniorCitizen         int64
Partner              object
Dependents           object
tenure                int64
PhoneService         object
MultipleLines        object
InternetService      object
OnlineSecurity       object
OnlineBackup         object
DeviceProtection     object
TechSupport          object
StreamingTV          object
StreamingMovies      object
Contract             object
PaperlessBilling     object
PaymentMethod        object
MonthlyCharges      float64
TotalCharges        float64
Churn                 int64
dtype: object
```

In [11]:
```
# Numerical columns for outlier detection
outlier_columns = [
    "tenure",
    "MonthlyCharges",
    "TotalCharges"
]

for col in outlier_columns:
    if col in df.columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)

        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        outliers = df[
            (df[col] < lower_bound) |
            (df[col] > upper_bound)
        ]

        print(f"{col}: {len(outliers)} outliers")
```

```
tenure: 0 outliers
MonthlyCharges: 0 outliers
TotalCharges: 0 outliers
```

In [12]:
```
for col in outlier_columns:
    if col in df.columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)

        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        df[col] = df[col].clip(
            lower_bound,
            upper_bound
        )

print("Outliers handled using IQR capping.")
```

```
Outliers handled using IQR capping.
```

In [13]:
```
# Average monthly charges based on total charges and tenure
df["AvgMonthlyCharges"] = np.where(
    df["tenure"] > 0,
    df["TotalCharges"] / df["tenure"],
    df["MonthlyCharges"]
)

# Service columns
service_columns = [
    "PhoneService",
    "MultipleLines",
    "InternetService",
    "OnlineSecurity",
```

```
    "OnlineBackup",
    "DeviceProtection",
    "TechSupport",
    "StreamingTV",
    "StreamingMovies"
]

# Count number of subscribed services
existing_services = [col for col in service_columns if col in df.columns]

df["TotalServices"] = 0

for col in existing_services:
    df["TotalServices"] += (
        df[col].astype(str).isin(["Yes", "DSL", "Fiber optic"])
    ).astype(int)

# Long-term customer feature
df["IsLongTermCustomer"] = (df["tenure"] >= 24).astype(int)

print("New features created successfully.")

display(df[[
    "tenure",
    "MonthlyCharges",
    "TotalCharges",
    "AvgMonthlyCharges",
    "TotalServices",
    "IsLongTermCustomer"
]].head())
```

```
New features created successfully.
```

	tenure	MonthlyCharges	TotalCharges	AvgMonthlyCharges	TotalServices	IsLongTermCust
0	1	29.85	29.85	29.850000	2	
1	34	56.95	1889.50	55.573529	4	
2	2	53.85	108.15	54.075000	4	
3	45	42.30	1840.75	40.905556	4	
4	2	70.70	151.65	75.825000	2	

In [14]:

```
from sklearn.preprocessing import StandardScaler

# Numerical features
scale_columns = [
    "tenure",
    "MonthlyCharges",
    "TotalCharges",
    "AvgMonthlyCharges",
    "TotalServices"
]

# Make sure only existing columns are selected
scale_columns = [
    col for col in scale_columns
    if col in df.columns
```

```
]

scaler = StandardScaler()

df_scaled = df.copy()

df_scaled[scale_columns] = scaler.fit_transform(
    df_scaled[scale_columns]
)

print("Numerical features standardized.")

display(df_scaled[scale_columns].head())
```

```
Numerical features standardized.
```

	tenure	MonthlyCharges	TotalCharges	AvgMonthlyCharges	
0	-1.277445	-1.160323	-0.994242	-1.156529	-0.9
1	0.066327	-0.259629	-0.173244	-0.304408	-0.0
2	-1.236724	-0.362660	-0.959674	-0.354049	-0.0
3	0.514251	-0.746535	-0.194766	-0.790302	-0.0
4	-1.236724	0.197365	-0.940470	0.366444	-0.9

In [15]:
```
from sklearn.model_selection import train_test_split

# Separate target
X = df.drop(columns=["Churn", "customerID"])

y = df["Churn"]

# Convert categorical variables into numerical variables
X = pd.get_dummies(X, drop_first=True)

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)

print("\nTraining target distribution:")
print(y_train.value_counts())

print("\nTesting target distribution:")
print(y_test.value_counts())
```

```
Training data shape: (5634, 33)
Testing data shape: (1409, 33)

Training target distribution:
Churn
0    4139
1    1495
Name: count, dtype: int64

Testing target distribution:
Churn
0    1035
1     374
Name: count, dtype: int64
```

In [16]:
```python
# Save cleaned dataset
df.to_csv(
    "Telecom_Customer_Churn_Cleaned.csv",
    index=False
)

print("Cleaned dataset exported successfully!")
print("File name: Telecom_Customer_Churn_Cleaned.csv")
```

```
Cleaned dataset exported successfully!
File name: Telecom_Customer_Churn_Cleaned.csv
```